

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_eb99bd9f-440\agent-  
a025b54ffd6f49da3.jsonl`  
- SHA-256: `eb56e9d5acb95a38c42cec642d99f2ca7c07727e65c52e6f07be4a9fae498684`  
- Source modified: `2026-07-06T06:11:42+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_eb99bd9f-440`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T06:09:59.892Z)

You are reviewing GitHub PR #35 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-b-domain-schema -> main).
It implements ADR 0011 (docs/adr/0011-provider-neutral-domain-schema.md): a provider-neutral domain schema (Track B). Changed files ONLY: src/tg_video_filter/db.py, src/tg_video_filter/models.py, tests/test_db.py, tests/test_delivery.py. Diff: +1299/-150.

Ground rules:

- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run the venv python for a targeted repro:
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. The DB layer is aiosqlite; use db.__apply_schema(conn) on an aiosqlite ":memory:" connection to build the schema, and db.upsert_account(...) to seed. A working repro is stronger evidence than prose.
- Compare against v1 behaviour on main when relevant: git -C C:/Users/avidu/Projects/Telegram show main:src/tg_video_filter/db.py
- Authoritative local gate ALREADY RUN (do not re-run the full suite): ruff check clean; pytest 434 passed @ 96.63% coverage (floor 80%); ruff format --check FAILS only on commands.py which is NOT in this PR (pre-existing on main) ? do not report that as a PR finding.
- Report ONLY defects that ship in THIS PR's diff. No style nits already passing ruff. No praise.
- Each finding needs a concrete failure scenario (inputs -> wrong result). Rank most-severe first.
- Severity: blocker (data loss/corruption/wrong core behaviour), high (real bug, narrow trigger), medium (latent/edge), low (hygiene). Be honest; downgrade if trigger is unrealistic.

ADVERSARIALLY VERIFY this single finding. Your job is to REFUTE it if you can ? read the exact code at the cited lines, trace the real control flow, and if useful run a repro with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe. Only return CONFIRMED if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. Return REFUTED if the code actually handles it, the trigger cannot occur, or it's pre-existing (not in this diff). Return PLAUSIBLE only if you cannot fully settle it. Also return the corrected severity.

FINDING (dimension migration):

title: Migration 003 INSERT OR IGNORE silently drops distinct v1 videos that share a fingerprint (tg:fp:) dedup_key
severity(claimed): high
file:line: src/tg_video_filter/db.py:539
summary: Two legitimately-distinct v1 videos (different channel/message, no document_id,

identical size/duration/height/width) map to the same tg:fp: dedup_key; UNIQUE(account_id, dedup_key) + INSERT OR IGNORE keeps only the first, so the second distinct row is lost from the live schema during migration.

failure_scenario: v1 enforced Tier-2 (fingerprint) dedup only at LOOKUP time (get_video_by_content_fingerprint); insert_video did NOT enforce it and there is no fingerprint unique index (v1 videos has only UNIQUE(user_id,channel_id,message_id) + the document_id partial index). Re-encodes with matching metadata are common, so a v1 DB can hold two distinct fingerprint-equal videos. Migration 003 computes `tg:fp:||size||:||dur||:||h||:||w\` for both (db.py:554-557); both collide on UNIQUE(account_id,dedup_key) (db.py:120-121); INSERT OR IGNORE (db.py:539) drops the second. Repro: two videos (channels -1001/-1002, msgs 1/2, no doc_id, all == 5000000/60/1080/1920); init_db -> videos_deprecated=2, content_items=1. The second video is unreachable via the live schema and lost after the deprecated table is dropped. evidence: src/tg_video_filter/db.py:539 (INSERT OR IGNORE), 551-558 (tg:fp branch), 120-121 (UNIQUE(account_id,dedup_key)); v1 `git show main:db.py` shows no fingerprint unique constraint. Repro output: 'Legacy videos: 2 (two distinct videos, same fingerprint) ... content_items: 1 ... DATA LOSS: 1 distinct video(s) dropped by INSERT OR IGNORE on tg:fp: collision'. recommendation: Fingerprint collisions among pre-existing rows are data, not duplicates. Either detect collisions and fall the losing rows back to the Tier-0 tg:msg: key (which is unique per source_id+message_id) so no row is dropped, or log/surface a count of dropped rows instead of silently IGNORE-ing. At minimum add a migration test seeding two fingerprint-equal videos and assert content_items count == videos count.

2. assistant (2026-07-06T06:10:04.570Z)

I'll verify this finding by reading the actual migration code and v1 schema, then running a repro.

3. user (2026-07-06T06:10:05.584Z)

```

500     Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
501     ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
502     DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
503     them), then backfills the three populated ones from the legacy
504     ``channels`` / ``videos`` / ``filter_config`` tables and renames the
505     legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
506     pattern). Idempotent: a no-op once the legacy tables have been renamed.
507     """
508     # Idempotency gate: if the legacy videos table is gone, this migration has
509     # already run (or the DB is fresh and _SCHEMA's legacy tables were already
510     # dropped by _apply_schema post-003). Nothing to backfill.
511     if not await _table_exists(conn, "videos"):
512         return
513
514     # 1. Backfill sources from channels. ADR 0009 backfill: already-seen
515     #   channels stay watched (watched=1). provider_source_id is the Telegram
516     #   channel_id (cast to TEXT for the provider-neutral column).
517     await conn.execute(
518         """
519         INSERT OR IGNORE INTO sources
520             (account_id, provider, provider_source_id, title, kind, watched,
521              seen_first_at, last_seen_at)
522         SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',
523                1, seen_first_at, seen_first_at
524         FROM channels
525         """
526     )
527
528     # 2. Backfill content_items from videos, computing dedup_key in pure SQL.
529     #   The three-branch decision tree (Option 1A):
530     #   a) document_id IS NOT NULL           ? 'tg:doc:' || document_id
531     #   b) all four fingerprint fields set   ? 'tg:fp:' || size:dur:h:w
532     #   c) else (Tier-0 fallback)           ? 'tg:msg:' || source_id:msg_id
533     #   The ``CASE`` is ordered strongest-first and each branch is guarded so
534     #   partial metadata never collapses to a degenerate key. source_id is
535     #   resolved by joining the channel's provider_source_id back to sources.id

```

```

536 # (Telegram message_id is per-channel, so source_id is required).
537 await conn.execute(
538     """
539     INSERT OR IGNORE INTO content_items
540     (account_id, source_id, provider, provider_message_id, content_type,
541     document_id, duration_s, width, height, size_bytes, mime_type,
542     dedup_key, matched, seen_at)
543     SELECT
544     v.user_id,
545     s.id,
546     'telegram',
547     CAST(v.message_id AS TEXT),
548     'video',
549     CASE WHEN v.document_id IS NOT NULL THEN CAST(v.document_id AS TEXT) END,
550     v.duration_s, v.width, v.height, v.size_bytes, v.mime_type,
551     CASE
552     WHEN v.document_id IS NOT NULL
553     THEN 'tg:doc:' || v.document_id
554     WHEN v.size_bytes IS NOT NULL AND v.duration_s IS NOT NULL
555     AND v.height IS NOT NULL AND v.width IS NOT NULL
556     THEN 'tg:fp:' || v.size_bytes || ':' || v.duration_s || ':'
557     || v.height || ':' || v.width
558     ELSE 'tg:msg:' || s.id || ':' || v.message_id
559     END,
560     v.matched,
561     v.seen_at
562     FROM videos v
563     JOIN sources s
564     ON s.account_id = v.user_id
565     AND s.provider = 'telegram'
566     AND s.provider_source_id = CAST(v.channel_id AS TEXT)
567     """
568 )
569
570 # 3. Backfill policies from filter_config: one 'default' policy per account
571 # (Option 2A). is_default=1 so the legacy load_filter_config shim reads
572 # this row. UNIQUE(account_id, name) dedups idempotent re-runs.
573 await conn.execute(
574     """
575     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
576     created_at, updated_at)
577     SELECT user_id, 'default', config_json, 1, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP
578     FROM filter_config
579     """
580 )
581
582 # 4. Rename (don't drop) the legacy tables ? kept for one release as a
583 # rollback safety net (ADR 0010 pattern).
584 await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
585 await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
586 await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
587
588
589 async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
590     """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
591     async with conn.execute(
592         "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
593     ) as cur:
594         return await cur.fetchone() is not None
595
596
597 # --- helpers -----
598
599
600 def _now() -> datetime:

```

```

601     """Timezone-aware UTC 'now', used for policy timestamps."""
602     return datetime.now(UTC)
603
604
605 def _row_to_account(row: aiosqlite.Row) -> Account:
606     return Account(
607         id=row["id"],
608         telegram_id=row["telegram_id"],
609         provider=row["provider"],
610         display_name=row["display_name"],
611         created_at=row["created_at"], # stored as ISO string; pydantic parses
612     )
613
614
615 def _row_to_telegram_session(row: aiosqlite.Row) -> TelegramSession:
616     return TelegramSession(
617         account_id=row["account_id"],
618         phone=row["phone"],
619         session_path=row["session_path"],

```

4. user (2026-07-06T06:10:06.440Z)

```

90     account_id      INTEGER NOT NULL REFERENCES accounts(id),
91     provider        TEXT NOT NULL DEFAULT 'telegram',
92     provider_source_id TEXT NOT NULL,
93     title           TEXT,
94     kind            TEXT NOT NULL DEFAULT 'channel',
95     watched         INTEGER NOT NULL DEFAULT 0,
96     seen_first_at   TEXT NOT NULL,
97     last_seen_at    TEXT,
98     UNIQUE (account_id, provider, provider_source_id)
99 );
100
101 CREATE TABLE IF NOT EXISTS content_items (
102     id                INTEGER PRIMARY KEY AUTOINCREMENT,
103     account_id        INTEGER NOT NULL REFERENCES accounts(id),
104     source_id         INTEGER NOT NULL REFERENCES sources(id),
105     provider          TEXT NOT NULL DEFAULT 'telegram',
106     provider_message_id TEXT NOT NULL,
107     content_type      TEXT NOT NULL DEFAULT 'video',
108     text              TEXT,
109     link_url          TEXT,
110     media_ref         TEXT,
111     document_id       TEXT,
112     duration_s        INTEGER,
113     width             INTEGER,
114     height            INTEGER,
115     size_bytes        INTEGER,
116     mime_type         TEXT,
117     dedup_key         TEXT NOT NULL,
118     matched           INTEGER NOT NULL DEFAULT 0,
119     seen_at           TEXT NOT NULL,
120     UNIQUE (account_id, source_id, provider_message_id),
121     UNIQUE (account_id, dedup_key)
122 );
123
124 CREATE TABLE IF NOT EXISTS policies (
125     id                INTEGER PRIMARY KEY AUTOINCREMENT,
126     account_id        INTEGER NOT NULL REFERENCES accounts(id),
127     name              TEXT NOT NULL,
128     config_json       TEXT NOT NULL,
129     is_default        INTEGER NOT NULL DEFAULT 0,
130     created_at        TEXT NOT NULL,
131     updated_at        TEXT NOT NULL,
132     UNIQUE (account_id, name)

```

```

133 );
134
135 CREATE TABLE IF NOT EXISTS destinations (
136     id            INTEGER PRIMARY KEY AUTOINCREMENT,
137     account_id    INTEGER NOT NULL REFERENCES accounts(id),
138     provider      TEXT NOT NULL DEFAULT 'telegram',
139     target        TEXT NOT NULL,
140     kind          TEXT NOT NULL DEFAULT 'saved',
141     title         TEXT,
142     is_output     INTEGER NOT NULL DEFAULT 0,
143     created_at    TEXT NOT NULL,
144     UNIQUE (account_id, provider, target)
145 );
146
147 CREATE TABLE IF NOT EXISTS routing_rules (
148     id            INTEGER PRIMARY KEY AUTOINCREMENT,
149     account_id    INTEGER NOT NULL REFERENCES accounts(id),
150     source_id     INTEGER NOT NULL REFERENCES sources(id),
151     policy_id     INTEGER REFERENCES policies(id),
152     destination_id INTEGER NOT NULL REFERENCES destinations(id),
153     enabled       INTEGER NOT NULL DEFAULT 1,
154     priority      INTEGER NOT NULL DEFAULT 0,
155     created_at    TEXT NOT NULL,
156     UNIQUE (account_id, source_id, destination_id)
157 );
158
159 CREATE TABLE IF NOT EXISTS delivery_attempts (
160     id            INTEGER PRIMARY KEY AUTOINCREMENT,
161     content_item_id INTEGER NOT NULL REFERENCES content_items(id),
162     destination_id INTEGER NOT NULL REFERENCES destinations(id),
163     route_id      INTEGER REFERENCES routing_rules(id),
164     status        TEXT NOT NULL DEFAULT 'pending',
165     attempt_count  INTEGER NOT NULL DEFAULT 0,
166     last_error    TEXT,
167     created_at    TEXT NOT NULL,
168     delivered_at  TEXT,
169     next_retry_at TEXT,
170     UNIQUE (content_item_id, destination_id)
171 );
172 ""
173
174 # Note: the ``idx_videos_user_document`` partial unique index is created by
175 # migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
176 # which is present on fresh tables but only added to legacy v1 DBs by the
177 # migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
178 # DBs that don't yet have the column.
179
180
181 async def init_db(db_path: str) -> None:
182     """Open the database at ``db_path`` and create/migrate the schema.
183
184     Safe to call on every startup. Applies the base schema (``IF NOT EXISTS``)
185     then runs any pending migrations via the ``user_version`` ladder.
186     """
187     async with aiosqlite.connect(db_path) as conn:
188         await _apply_schema(conn)
189
190
191 async def _apply_schema(conn: aiosqlite.Connection) -> None:
192     # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
193     # and a busy timeout so concurrent access waits instead of failing immediately.
194     await conn.execute("PRAGMA journal_mode = WAL")
195     await conn.execute("PRAGMA foreign_keys = ON")
196     await conn.execute("PRAGMA busy_timeout = 5000")
197     await conn.executescript(_SCHEMA)

```

```

198     # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
199     # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
200     # migration 002's FK rebuild and migration 003's backfill on legacy DBs).
201     # Once migration 003 has run (signalled by ``channels_deprecated`` existing),
202     # any *empty* recreations of those legacy tables from a subsequent
203     # ``init_db`` are dropped here so they don't shadow the renamed copies.
204     if await _table_exists(conn, "channels_deprecated"):
205         await conn.execute("DROP TABLE IF EXISTS channels")
206         await conn.execute("DROP TABLE IF EXISTS videos")
207         await conn.execute("DROP TABLE IF EXISTS filter_config")
208     await _apply_migrations(conn)
209     await conn.commit()
210
211
212     # --- migrations -----
213     #
214     # Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
215     # Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
216     # the source of truth for *fresh* databases; the migrations below reconcile
217     # *existing* databases that were created by an older schema version.
218     #
219     # To add a new migration:
220     #   1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
221     #   2. Implement ``async def _migration_NN(conn) -> None`` (must be idempotent).
222     #   3. Add a test in ``tests/test_db.py`` that seeds an old schema, runs
223     #       ``init_db``, and asserts the new structure.
224
225     _MIGRATIONS: list[tuple[int, Callable[[aiosqlite.Connection], Awaitable[None]]]] = []
226
227
228     def _migration(
229         version: int,

```

5. assistant (2026-07-06T06:10:08.866Z)

Now let me check the v1 schema on main and run a repro.

6. user (2026-07-06T06:10:12.281Z)

```

30:     telegram_id  INTEGER UNIQUE,
31:     provider      TEXT NOT NULL DEFAULT 'telegram',
32:     display_name  TEXT,
33:     created_at    TEXT NOT NULL
34-);
35-
36-CREATE TABLE IF NOT EXISTS telegram_sessions (
37:     account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
38:     phone         TEXT,
39:     session_path  TEXT NOT NULL,
40:     enabled       INTEGER NOT NULL DEFAULT 1,
41:     created_at    TEXT NOT NULL,
42:     last_login_at TEXT
43-);
44-
45-CREATE TABLE IF NOT EXISTS channels (
46:     id            INTEGER PRIMARY KEY AUTOINCREMENT,
47:     user_id       INTEGER NOT NULL REFERENCES accounts(id),
48:     channel_id    INTEGER NOT NULL,
49:     title         TEXT,
50:     seen_first_at TEXT NOT NULL,
51:     UNIQUE (user_id, channel_id)
52-);
53-
54-CREATE TABLE IF NOT EXISTS videos (
55:     id            INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

56-     user_id      INTEGER NOT NULL REFERENCES accounts(id),
57-     channel_id   INTEGER NOT NULL,
58-     message_id   INTEGER NOT NULL,
59-     document_id  INTEGER,
60-     duration_s   INTEGER,
61-     width        INTEGER,
62-     height       INTEGER,
63-     size_bytes   INTEGER,
64-     mime_type    TEXT,
65-     matched      INTEGER NOT NULL DEFAULT 0,
66-     forwarded    INTEGER NOT NULL DEFAULT 0,
67-     seen_at      TEXT NOT NULL,
68-     UNIQUE (user_id, channel_id, message_id)
69-);
70-
71-CREATE TABLE IF NOT EXISTS filter_config (
72-     user_id      INTEGER PRIMARY KEY REFERENCES accounts(id),
73-     config_json  TEXT NOT NULL
74-);
75-""
76-
77-# Note: the `idx_videos_user_document` partial unique index is created by
78-# migration 001 (below), not in `_SCHEMA`. It references `document_id`
79-# which is present on fresh tables but only added to legacy v1 DBs by the
80-# migration; putting it in `_SCHEMA` would break `executescript` on legacy
81-# DBs that don't yet have the column.
82-
83-
84-async def init_db(db_path: str) -> None:
85-     """Open the database at `db_path` and create/migrate the schema.
86-
87-     Safe to call on every startup. Applies the base schema (`IF NOT EXISTS`)
88-     then runs any pending migrations via the `user_version` ladder.
89-     """
90-     async with aiosqlite.connect(db_path) as conn:
91-         await _apply_schema(conn)
92-
93-
94-async def _apply_schema(conn: aiosqlite.Connection) -> None:
95-     # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
96-     # and a busy timeout so concurrent access waits instead of failing immediately.
97-     await conn.execute("PRAGMA journal_mode = WAL")
98-     await conn.execute("PRAGMA foreign_keys = ON")
99-     await conn.execute("PRAGMA busy_timeout = 5000")
100-    await conn.executescript(_SCHEMA)
101-    await _apply_migrations(conn)
102-    await conn.commit()
103-
104-
105-    """CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
106-    "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
107-    """
108-
109-
110-@migration(2)

```

7. assistant (2026-07-06T06:10:17.242Z)

The v1 schema confirms: `videos` has only `UNIQUE(user\_id, channel\_id, message\_id)` plus a partial unique index on `document\_id`. No fingerprint unique constraint. So two distinct videos with different (channel_id, message_id), no document_id, and identical size/duration/height/width can coexist in v1. Let me verify v1's `insert\_video` doesn't enforce fingerprint dedup, then run the repro.

8. user (2026-07-06T06:10:20.909Z)

```

670:async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
671-    """Insert a new video row. Raises IntegrityError on a dedup-key collision.
672-
673-    Two dedup constraints protect against duplicates:
674-    - ``UNIQUE (user_id, channel_id, message_id)`` ? same message instance.
675-    - ``UNIQUE (user_id, document_id) WHERE document_id IS NOT NULL`` ?
676-      same file forwarded across channels (content dedup Tier 1).
677-    """
678-    cur = await conn.execute(
679-        """INSERT INTO videos
680-            (user_id, channel_id, message_id, document_id, duration_s, width, height,
681-             size_bytes, mime_type, matched, forwarded, seen_at)
682-            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
683-        (
684-            video.user_id,
685-            video.channel_id,
686-            video.message_id,
687-            video.document_id,
688-            video.duration_s,
689-            video.width,
690-            video.height,

```

9. assistant (2026-07-06T06:10:25.673Z)

Confirmed: v1 `insert\_video` only enforces the two constraints (message-instance and document_id). Fingerprint dedup is NOT enforced at insert time in v1. So two fingerprint-equal videos with no document_id CAN coexist. Now the repro.

10. user (2026-07-06T06:10:37.837Z)

```

Exit code 1
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\repro.py", line 2, in <module>
    from src.tg_video_filter import db
ModuleNotFoundError: No module named 'src'

```

11. user (2026-07-06T06:10:44.935Z)

src/tg_video_filter/__init__.py

Tell hatchling where the package lives (src layout).

```

[tool.hatch.build.targets.wheel]
packages = ["src/tg_video_filter"]

```

--- pytest -----

```
[tool.pytest.ini_options]
```

12. assistant (2026-07-06T06:10:49.109Z)

The package installs as `tg\_video\_filter`. Let me fix the import.

13. user (2026-07-06T06:10:52.918Z)

```

Legacy videos: 2
content_items: 1
  tg:fp:5000000:60:1080:1920 1
DATA LOSS (1 dropped)

```

14. assistant (2026-07-06T06:11:03.688Z)

The repro confirms the data loss precisely: 2 distinct legacy videos with no document_id and identical fingerprint collapse to 1 content_item. The second video (channel -1002, message 2) is silently dropped by `INSERT OR IGNORE` on the `UNIQUE(account\_id, dedup\_key)` collision.

Let me verify the two videos are genuinely distinct in v1 (survive v1's constraints) ? they have different (channel_id, message_id) so v1 message-instance UNIQUE is satisfied, and no document_id so the partial index doesn't apply. Both are legitimately live in v1. The repro already seeded both successfully into `videos` (count=2), confirming this.

Now let me assess whether the trigger is realistic. The finding claims re-encodes/reposts with matching metadata are common. Two videos with byte-identical `size\_bytes` AND identical duration/width/height but no `document\_id` ? this requires the exact same file size in bytes. That's the same underlying file (identical bytes ? identical size). If it's the same file, Telegram would normally give it the same `document\_id`, in which case branch (a) `tg:doc:` fires and no collision. The collision only occurs when `document\_id IS NULL` for both.

Let me check whether v1 records typically have `document\_id` populated, which affects how realistic the NULL-document_id + identical-fingerprint case is.

15. user (2026-07-06T06:11:09.546Z)

```
668-
669-
670:async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:
671-    """Insert a new video row. Raises IntegrityError on a dedup-key collision.
672-
673-    Two dedup constraints protect against duplicates:
674-    - ``UNIQUE (user_id, channel_id, message_id)`` ? same message instance.
675-    - ``UNIQUE (user_id, document_id) WHERE document_id IS NOT NULL`` ?
676-      same file forwarded across channels (content dedup Tier 1).
677-    """
678-    cur = await conn.execute(
679-        """INSERT INTO videos
680-           (user_id, channel_id, message_id, document_id, duration_s, width, height,
681-            size_bytes, mime_type, matched, forwarded, seen_at)
682-           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
683-        (
684-            video.user_id,
685-            video.channel_id,
686-            video.message_id,
687-            video.document_id,
688-            video.duration_s,
689-            video.width,
690-            video.height,
691-            video.size_bytes,
692-            video.mime_type,
693-            video.matched,
694-            video.forwarded,
695-            video.seen_at,
696-        )
697-    )
698-    return video
699-
700-
701-
702-
703-
704-
705-
706-
707-
708-
709-
710-
711-
712-
713-
714-
715-
716-
717-
718-
719-
720-
721-
722-
723-
724-
725-
726-
727-
728-
729-
730-
731-
732-
733-
734-
735-
736-
737-
738-
739-
740-
741-
742-
743-
744-
745-
746-
747-
748-
749-
750-
751-
752-
753-
754-
755-
756-
757-
758-
759-
760-
761-
762-
763-
764-
765-
766-
767-
768-
769-
770-
771-
772-
773-
774-
775-
776-
777-
778-
779-
780-
781-
782-
783-
784-
785-
786-
787-
788-
789-
790-
791-
792-
793-
794-
795-
796-
797-
798-
799-
800-
801-
802-
803-
804-
805-
806-
807-
808-
809-
810-
811-
812-
813-
814-
815-
816-
817-
818-
819-
820-
821-
822-
823-
824-
825-
826-
827-
828-
829-
830-
831-
832-
833-
834-
835-
836-
837-
838-
839-
840-
841-
842-
843-
844-
845-
846-
847-
848-
849-
850-
851-
852-
853-
854-
855-
856-
857-
858-
859-
860-
861-
862-
863-
864-
865-
866-
867-
868-
869-
870-
871-
872-
873-
874-
875-
876-
877-
878-
879-
880-
881-
882-
883-
884-
885-
886-
887-
888-
889-
890-
891-
892-
893-
894-
895-
896-
897-
898-
899-
900-
901-
902-
903-
904-
905-
906-
907-
908-
909-
910-
911-
912-
913-
914-
915-
916-
917-
918-
919-
920-
921-
922-
923-
924-
925-
926-
927-
928-
929-
930-
931-
932-
933-
934-
935-
936-
937-
938-
939-
940-
941-
942-
943-
944-
945-
946-
947-
948-
949-
950-
951-
952-
953-
954-
955-
956-
957-
958-
959-
960-
961-
962-
963-
964-
965-
966-
967-
968-
969-
970-
971-
972-
973-
974-
975-
976-
977-
978-
979-
980-
981-
982-
983-
984-
985-
986-
987-
988-
989-
990-
991-
992-
993-
994-
995-
996-
997-
998-
999-
1000-
```

16. assistant (2026-07-06T06:11:19.481Z)

`document\_id` is `int | None` and defaults to None ? it can genuinely be absent (e.g., media Telethon couldn't fully resolve). So NULL document_id videos are a real class of data.

The key question for severity: how do two DISTINCT videos end up with identical (size_bytes, duration_s, height, width) but BOTH have NULL document_id?

- The tg:fp fingerprint is a heuristic that intentionally treats byte-identical-metadata videos as the same content (that's its whole purpose as Tier-2 dedup). But v1 only applied this at *lookup* time, never enforced at insert. So v1 could and did store both rows.
- Migration 003 retroactively applies Tier-2 dedup to historical data via the UNIQUE constraint.

Where the fingerprint genuinely collides on *distinct* content (two different videos that happen to share size/duration/dimensions ? plausible for short clips, re-encodes with deterministic encoders, or coincidental collisions), the migration drops a real row.

The behavior is a genuine consequence of the diff. Whether the second row is "the same content" or "distinct content" depends on the fingerprint's accuracy ? but v1 preserved both rows and the migration destroys one, silently, with no logging. The row becomes unreachable and is lost when `videos\_deprecated` is eventually dropped.

Let me confirm there's no logging/count surfacing of ignored rows in the migration.

17. user (2026-07-06T06:11:20.886Z)

```
203: # ``init_db`` are dropped here so they don't shadow the renamed copies.
369: # (The empty accounts table created by _SCHEMA on a legacy DB is dropped
482: # against the legacy videos table) was dropped along with the legacy copy
510: # dropped by _apply_schema post-003). Nothing to backfill.
519:     INSERT OR IGNORE INTO sources
539:     INSERT OR IGNORE INTO content_items
575:     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
1252:     ``forwarded`` is always ``False`` post-ADR-0011 (the column is dropped;
```

18. assistant (2026-07-06T06:11:30.638Z)

Confirmed: no logging, no count surfacing, no collision detection. The `INSERT OR IGNORE` at line 539 silently drops the losing row. The finding is accurate on every mechanical point:

1. v1 schema (main) has no fingerprint unique constraint ? verified in git show.
2. v1 `insert\_video` doesn't enforce fingerprint dedup ? verified.
3. Migration 003 computes `tg:fp:` key at db.py:554-557 ? verified.
4. `UNIQUE(account\_id, dedup\_key)` at db.py:120-121 ? verified.
5. `INSERT OR IGNORE` at db.py:539 ? verified.
6. Repro reproduces exactly the described data loss (2 ? 1).

This ships in this PR's diff (migration 003 is new in the ADR 0011 work).

On severity: The claimed severity is "high". This is a migration that silently drops a real, previously-live row with no logging. It's data loss on a user's historical DB. However, the trigger requires two genuinely-distinct videos to have byte-identical size AND identical duration/height/width AND both lack a document_id. That's a real but narrow intersection ? most videos carry a document_id (branch a fires first, avoiding the collision), and byte-identical size across genuinely-different content is uncommon though not impossible (short clips, re-encodes). When it does occur, the loss is permanent and silent. This fits "high" (real bug, narrow trigger) rather than blocker. I'll confirm at high.

19. user (2026-07-06T06:11:42.863Z)

Structured output provided successfully